Character Frequency Analysis

Prepared by: Rene Andre Bedonia Jocsing

Published: October 16, 2025 **Deadline:** November 27, 2025

This laboratory assignment combines string processing, character frequency analysis, and data structure manipulation using both C and Assembly. You will implement a program that analyzes text input to find modal (most frequently occurring) characters, displaying comprehensive statistics.

Background

Character frequency analysis is fundamental in many applications including cryptography, compression algorithms, and natural language processing. This exercise demonstrates efficient string processing techniques, specifically in modal character detection.

Learning Objectives

By the end of this laboratory exercise, students should be able to:

- Implement string traversal and character counting algorithms in assembly
- Apply proper calling conventions for C-Assembly interfacing
- Implement data structure operations (arrays, structs) across language boundaries
- Perform statistical analysis on character frequency data
- Handle null-terminated strings and array bounds checking
- Implement modal value detection algorithms

Task

Create a program that performs comprehensive character frequency analysis:

1. Prompt the user to input a text string
2. Count frequency of each character (excluding spaces)
3. Find and display **all** modal (most frequent) characters
4. Display complete frequency statistics for all characters
5. Show summary statistics (total characters analyzed, modal frequency)

Data Structure

Use a C struct for storing character data. For example:

```c
typedef struct ModalCharacter {
    char character;
    int times;
} modal_character;
```

Implementation Details

- Ignore space characters in frequency analysis
- Null-terminate arrays as per the C convention
- Display both individual character frequencies and modal characters
- You may **interface C and Assembly**. However, the **number of lines in your C source code must not exceed the number of lines in your Assembly source code.**

Expected Output

```
Input a string: programming is fun and challenging
Character frequencies:
a: 3
c: 1
d: 1
e: 1
f: 1
g: 4
h: 1
i: 3
l: 2
m: 2
n: 5
o: 1
p: 1
r: 2
s: 1
u: 1
Modal character(s) with frequency 5:
n: 5
Total unique characters analyzed: 17
Total characters analyzed: 34
Modal frequency: 5
```

Testing

Verify your implementation handles:

- Single modal character
- Multiple modal characters with same frequency
- Strings with various character distributions
- Edge cases: empty strings, single characters, all same characters
- Proper space character exclusion

Laboratory Defense

The instructor will be available during scheduled laboratory hours in the assigned room for defense or consultation. You will be catered to at a first-come first-served basis. It is mandatory to present your code, answer inquiries, and perform live programming if the instructor deems it necessary to further check your understanding.

If a laboratory defense for an activity fails to be conducted within **one month** of its assignment, the instructor will be forced to give a failing grade. Scoring will be done during the laboratory defense (no defense, no grade policy). Extensions may be granted due to extraordinary circumstances, but ultimately remains up to the judgment of the instructor.

Academic Honesty

The usage of Large Language Models (e.g. ChatGPT, Claude, Deepseek, etc.) to generate code is considered cheating. As cheating is against the university's code of ethics, it is subject to failure in the course and harsh disciplinary action.

Rubric for Programming Exercises (50 pts)

Criteria (10 Points Each)	Excellent (9-10)	Good (6-8)	Fair (3-5)	Poor (0-2)
Program Correctness	Program executes correctly with no syntax or runtime errors, meets/exceeds specifications, and displays correct output	Program executes and outputs with minor errors, yet meets specifications	Program executes and outputs with major errors, yet somehow meets specifications	Program does not execute or does not meet specs
Logical Design	Program is logically well-designed with excellent structure and flow	Program has slight logic errors that do not significantly affect the results	Program has significant logic errors affecting functionality	Program logic is fundamentally incorrect
Code Mastery	Programmer demonstrates excellent mastery over the program's code	Programmer demonstrates adequate mastery over the program's code	Programmer demonstrates fair mastery over the program's code	Programmer demonstrates poor mastery over the program's code

Criteria (10 Points Each)	Excellent (9-10)	Good (6-8)	Fair (3-5)	Poor (0-2)
Engineering Standards	Program is stylistically well designed from an engineering standpoint	Slight inappropriate design choices (i.e., poor variable names, improper indentation)	Severe inappropriate design choices (i.e., code repetition, redundancy)	Program is poorly written
Documentation*	Program is well-documented: comments exist for clarity, not redundancy	Missing one required comment or some redundant comments	Missing two or more required comments or many redundant comments	Most documentation missing or most documentation is redundant

***Remember: "Code tells you how, comments tell you why."** — Jeff Atwood, co-founder of Stack Overflow and Discourse

---

See the laboratory manual for submission requirements.